

Security Door Access System Software Project

Christine Marie Lirazan

Department of Mechatronics Engineering, Kennesaw State University

clirazan@students.kennesaw.edu

Abstract—This project presents a Security Door Access System developed in Python using TCP socket communication. The system allows users to enter an Employee ID and PIN, verifies the information through a server, and records access attempts. Security features such as automatic relocking and account lockout were added to improve the functionality of the system.

Keywords—Python, TCP socket communication, client-server architecture, access control, authentication, network programming, security system

I. INTRODUCTION

This project demonstrates the use of networking concepts to create a simulated building access system. A client-server structure was used to allow communication between an entry device and an authentication server. The project applies TCP communication concepts to a practical security application.

II. METHODOLOGY

The system was developed in Python using the socket library for TCP communication. The implementation was based on the provided code from the Application and Transport Layer Plus Modbus Modification Assignment.

The client collects an Employee ID and PIN and sends the information to the server. The server verifies the credentials using a JSON database and returns either ACCESS GRANTED or ACCESS DENIED. Additional functions include

automatic door locking after three seconds, account lockout after three failed attempts, and saving login records with timestamps.

III. CODE LISTING

A. server.py

```
"""
```

Class: MTRE 4410

Section: W01

Term: Summer 2026

Instructor: Razvan Voicu

Name: Christine Marie Lirazan

Project: Software Project

File: server.py

Description:

Receives employee credentials from the client, verifies access, sends the authentication result back to the client, and logs each login attempt.

```
"""
```

```
import socket
```

```
import json
```

```
import time
```

```
from datetime import datetime
```

```
# Real-World Mapping:
```

```
# This program simulates a building security door access server.
```

```
# The server receives an Employee ID and PIN from the client,
```

```
# verifies the credentials, and grants or denies access.
```

```
HOST = "127.0.0.1"
```

```
PORT = 5055
```

```

# Requirement: Receiver Processing -
Unpack the message and act on it
# (display, save, control a mock actuator).
# Load employee database.
with open("users.json", "r") as file:
    users = json.load(file)

# Extra Feature: Store failed login attempts
for each employee.
failed_attempts = {}

# Requirement: Communication Layer -
Package and send the input over a
# communication channel. (Recommend
Sockets)
server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

# Requirement: Client/Server Architecture -
Demonstrate separation of
# concerns—client process vs. server
process.
server.bind((HOST, PORT))
server.listen()

print("❀" * 50)
print("      SECURITY DOOR ACCESS
SERVER")
print("❀" * 50)
print("Server is running...")
print("Waiting for client connection...\n")

while True:

    # Requirement: Server - Listens on a port.
    client, address = server.accept()

    print(f'Client connected from {address}')

    # Requirement: Server - Receives and
    parses the incoming frame.
    data = client.recv(1024).decode()

    employee, pin = data.split(",")

    # Create login counter for new employees

```

```

    if employee not in failed_attempts:
        failed_attempts[employee] = 0

    # Extra Feature: Lock account after three
    failed login attempts.
    if failed_attempts[employee] >= 3:

        response = (
            "ACCOUNT LOCKED ჯო'□`ჯს\n"
            "Too many failed login attempts.\n"
            "Please contact the administrator."
        )

    # Requirement: Receiver Processing -
    Unpack the message and act on it
    # (display, save, control a mock actuator).
    elif employee in users and
    users[employee] == pin:

        failed_attempts[employee] = 0

        response = (
            "ACCESS GRANTED ჯო^□^ჯს\n"
            "Door Unlocked"
        )

        print(f'Employee {employee}
authenticated.')
        print("Door Unlocked")

    # Extra Feature: Automatically lock the
    door after 3 seconds.
    time.sleep(3)

    print("Door Locked\n")

else:

    failed_attempts[employee] += 1

    remaining = 3 -
    failed_attempts[employee]

    if remaining == 0:

        response = (

```

```

        "ACCOUNT LOCKED
        "\n"
        "Too many failed login attempts."
    )

    else:

        response = (
            f"ACCESS DENIED \n"
            f"Attempts Remaining:
            {remaining}"
        )

        # Requirement: Receiver Processing -
        Unpack the message and act on it
        # (display, save, control a mock actuator).
        client.send(response.encode())

        # Extra Feature: Record every login
        attempt with a timestamp.
        with open("access_log.txt", "a") as log:

            log.write(
                f"{datetime.now()} | "
                f"Employee: {employee} | "
                f"{response.replace(chr(10), ' ')}\n"
            )

        client.close()
B. client.py
"""

```

Class: MTRE 4410
 Section: W01
 Term: Summer 2026
 Instructor: Razvan Voicu
 Name: Christine Marie Lirazan
 Project: Software Project
 File: client.py

Description:
 Captures an Employee ID and PIN from the user,
 sends the credentials to the server using a TCP socket,
 and displays the authentication result.

```

"""

import socket

# Real-World Mapping:
# This program simulates a building security
# door keypad.
# An employee enters an Employee ID and
# PIN, which are
# sent to the building's access control server.

HOST = "127.0.0.1"
PORT = 5055

# Extra Feature: Welcome screen.
print(" * 55)
print("      SECURITY DOOR ACCESS
SYSTEM")
print(" * 55)
print("Welcome \n")
print("Please enter your credentials
below.\n")

# Requirement: Input Acquisition - Read
data from a simulated or real source
# (keyboard, file, sensor).
employee = input("Employee ID: ")

# Requirement: Input Acquisition - Read
data from a simulated or real source
# (keyboard, file, sensor).
pin = input("PIN: ")

# Requirement: Communication Layer -
Package and send the input over a
# communication channel. (Recommend
Sockets)
client = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

# Requirement: Client/Server Architecture -
Demonstrate separation of
# concerns—client process vs. server
process.
client.connect((HOST, PORT))

```

```
# Requirement: Client - Sends the data in a
structured frame.
message = employee + "," + pin
```

```
# Requirement: Communication Layer -
Package and send the input over a
# communication channel. (Recommend
Sockets)
client.send(message.encode())
```

```
# Requirement: Receiver Processing -
Unpack the message and act on it
# (display, save, control a mock actuator).
response = client.recv(1024).decode()
```

```
print("\n" + "❀" * 55)
print("Authentication Result")
print("-" * 55)
print(response)
print("❀" * 55)
```

```
client.close()
```

```
print("\nConnection closed.")
```

```
C. users.json
```

```
{
  "1001": "0720",
  "1002": "0909",
  "1003": "9999",
  "1004": "0806",
  "1005": "6769"
}
```

IV. RESULTS

The system successfully communicated between the client and server and correctly processed login requests. Valid credentials allowed access, while incorrect credentials were rejected. The account lockout feature activated after three failed attempts, and all login activity was recorded in the access log.

A. Figures

```
(base) tingarys-MacBook-Pro Lirazan_Software Project % python3 server.py
=====
SECURITY DOOR ACCESS SERVER
=====
Server is running...
Waiting for client connection...
```

Fig. 1. Server Running in Terminal

```
(base) tingarys-MacBook-Pro Lirazan_Software Project % python3 client.py
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: █
```

Fig. 2. Client Welcome Screen

```
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: 1001
PIN: 0720
Authentication Result:
ACCESS GRANTED <w> - <w>
Door Unlocked
=====
Connection closed.
(base) tingarys-MacBook-Pro Lirazan_Software Project % █
```

Fig. 3. Successful Login (Access Granted)

```
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: 1001
PIN: 0720
Authentication Result:
ACCESS GRANTED <w> - <w>
Door Unlocked
=====
Connection closed.
(base) tingarys-MacBook-Pro Lirazan_Software Project % python3 client.py
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: 1001
PIN: 0720
Authentication Result:
ACCESS GRANTED <w> - <w>
Door Unlocked
=====
Connection closed.
(base) tingarys-MacBook-Pro Lirazan_Software Project % █
```

Fig. 4. Failed Login (Access Denied)

```
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: 1001
PIN: 0720
Authentication Result:
ACCESS GRANTED <w> - <w>
Door Unlocked
=====
Connection closed.
(base) tingarys-MacBook-Pro Lirazan_Software Project % python3 client.py
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: 1001
PIN: 0720
Authentication Result:
ACCESS GRANTED <w> - <w>
Door Unlocked
=====
Connection closed.
(base) tingarys-MacBook-Pro Lirazan_Software Project % python3 client.py
=====
SECURITY DOOR ACCESS SYSTEM
=====
Welcome <w> - <w>
Please enter your credentials below.
Employee ID: 1001
PIN: 0720
Authentication Result:
ACCESS GRANTED <w> - <w>
Door Unlocked
=====
Connection closed.
(base) tingarys-MacBook-Pro Lirazan_Software Project % █
```

Fig. 5. Account Locked After Three Failed Attempts

```
(base) tingarys-MacBook-Pro Lirazan_Software Project % cat access_log.txt
2026-07-21 23:07:04.719662 | Employee: 1001 | ACCESS GRANTED <w> - <w> Door Unlocked
2026-07-21 23:07:30.538464 | Employee: 1001 | ACCESS DENIED <w> - <w> Attempts Remaining: 2
2026-07-21 23:07:46.608431 | Employee: 1001 | ACCESS DENIED <w> - <w> Attempts Remaining: 1
2026-07-21 23:07:50.313601 | Employee: 1001 | ACCOUNT LOCKED <w> - <w> Too many failed login attempt s.
2026-07-21 23:18:53.076611 | Employee: 1001 | ACCESS GRANTED <w> - <w> Door Unlocked
2026-07-21 23:18:46.070608 | Employee: 1001 | ACCESS DENIED <w> - <w> Attempts Remaining: 2
2026-07-21 23:21:12.920386 | Employee: 1001 | ACCESS DENIED <w> - <w> Attempts Remaining: 1
2026-07-21 23:21:26.525985 | Employee: 1001 | ACCOUNT LOCKED <w> - <w> Too many failed login attempt s.
(base) tingarys-MacBook-Pro Lirazan_Software Project % █
```

Fig. 6. Access Log with Timestamped Login Attempts

V. CONCLUSION

In all, a Security Door Access System was successfully implemented using Python and TCP socket communication. The project showed how networking principles can be applied to a real-world security system.

VI. REFERENCES

[1] R. Voicu, "Application and Transport Layer Plus Modbus Modification

Assignment,” Department of Robotics & Mechatronics Engineering, Kennesaw State University, Course materials provided through D2L, 2026.

- [2] Python Software Foundation, “socket — Low-level networking interface,” Python Documentation. [Online]. Available:
<https://docs.python.org/3/library/socket.html>